

```
In [37]: ▶ import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import plot_tree
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.tree import export_text
from sklearn.metrics import r2_score
import warnings
warnings.filterwarnings("ignore")
```

```
In [3]: ▶ df = pd.read_csv("C:\\Users\\sd\\OneDrive\\Desktop\\Mall_Customers_1.csv")
```

```
In [4]: ▶ df
```

```
Out[4]:
```

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40
...	...	...	...	...	...
195	196	Female	35	120	79
196	197	Female	45	126	28
197	198	Male	32	126	74
198	199	Male	32	137	18
199	200	Male	30	137	83

200 rows × 5 columns

Data processing

```
In [5]: df.describe()
```

```
Out[5]:
```

	CustomerID	Age	Annual Income (k\$)	Spending Score (1-100)
count	200.000000	200.000000	200.000000	200.000000
mean	100.500000	38.850000	60.560000	50.200000
std	57.879185	13.969007	26.264721	25.823522
min	1.000000	18.000000	15.000000	1.000000
25%	50.750000	28.750000	41.500000	34.750000
50%	100.500000	36.000000	61.500000	50.000000
75%	150.250000	49.000000	78.000000	73.000000
max	200.000000	70.000000	137.000000	99.000000

Handling Missing Values: Effective handling might involve imputation techniques (like mean, median, or using machine learning algorithms to predict missing values) if any of these features contain missing values.

```
In [6]: df.isnull().sum()
```

```
Out[6]: CustomerID      0
Gender      0
Age      0
Annual Income (k$)      0
Spending Score (1-100)  0
dtype: int64
```

Categorical Variable Encoding: There is one variable called Gender that needed to be changed into numerical terms.

```
In [7]: binary_columns = ['Gender']
le = LabelEncoder()
for col in binary_columns:
    df[col] = le.fit_transform(df[col])
```

In [8]: `df`

Out[8]:

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	1	19	15	39
1	2	1	21	15	81
2	3	0	20	16	6
3	4	0	23	16	77
4	5	0	31	17	40
...	...	...	...	...	...
195	196	0	35	120	79
196	197	0	45	126	28
197	198	1	32	126	74
198	199	1	32	137	18
199	200	1	30	137	83

200 rows × 5 columns

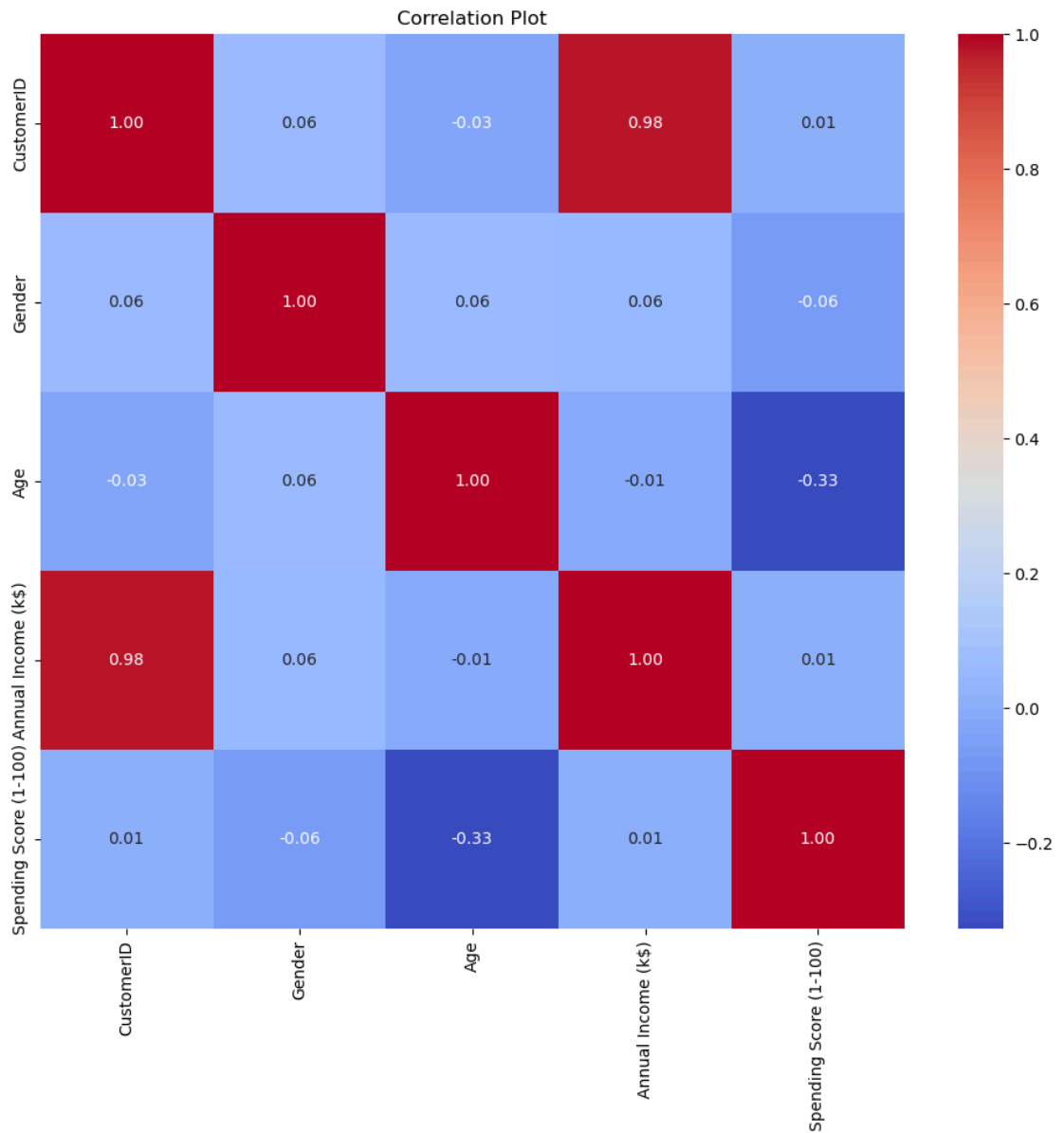
In [9]: `print(df.info())`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   CustomerID                           200 non-null   int64
1   Gender                               200 non-null   int32
2   Age                                   200 non-null   int64
3   Annual Income (k$)                   200 non-null   int64
4   Spending Score (1-100)               200 non-null   int64
dtypes: int32(1), int64(4)
memory usage: 7.2 KB
None
```

EDA

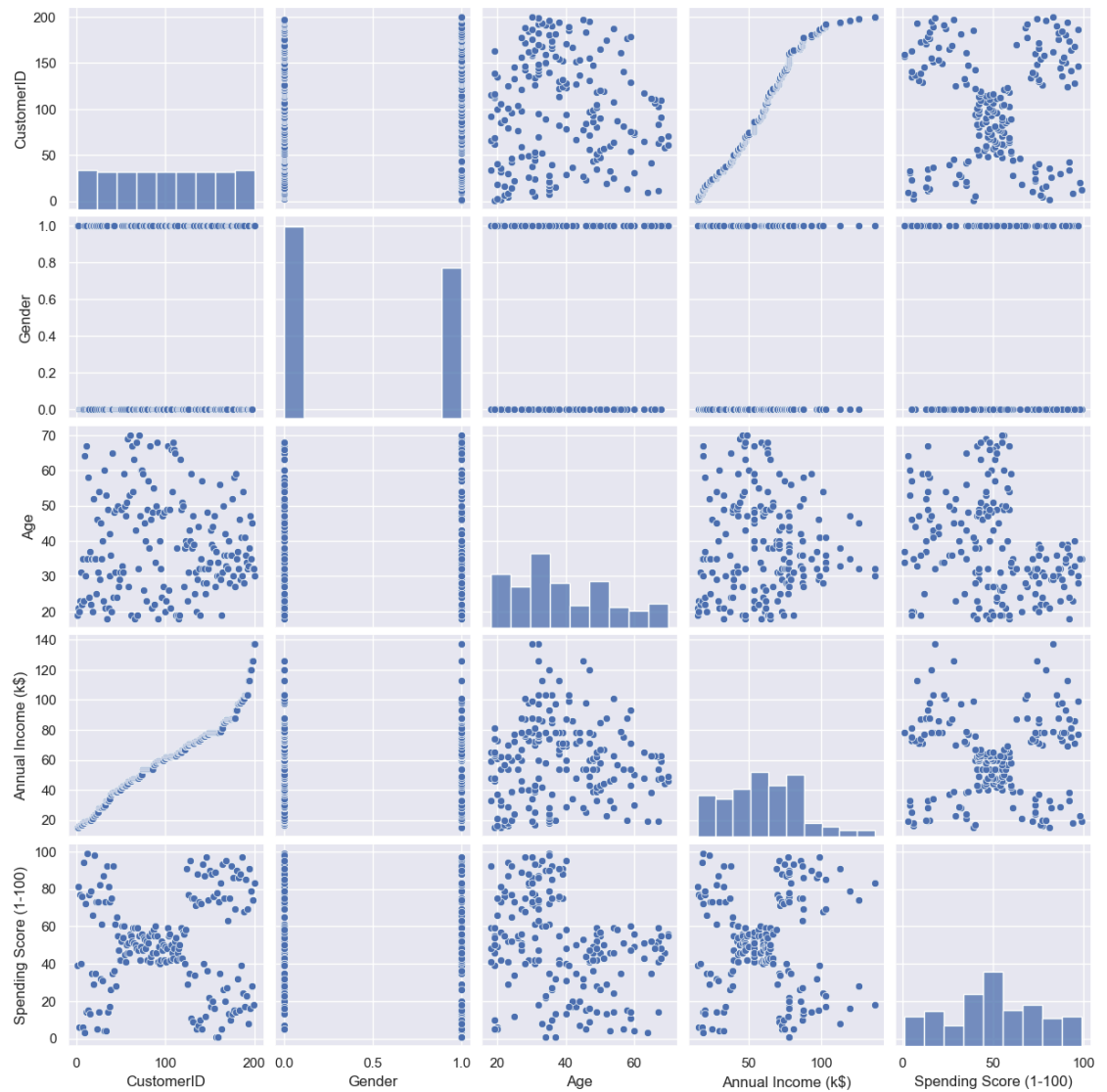
exploring the distribution of spending scores across different age groups or income brackets could reveal interesting patterns. identifying correlations or clusters within these features. For example, finding that individuals in certain age groups with higher income tend to have a higher spending score could be an insightful pattern for marketing strategies.

```
In [10]: #plotting correlation matrix
correlation_matrix = df.corr()
plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Correlation Plot")
plt.show()
```



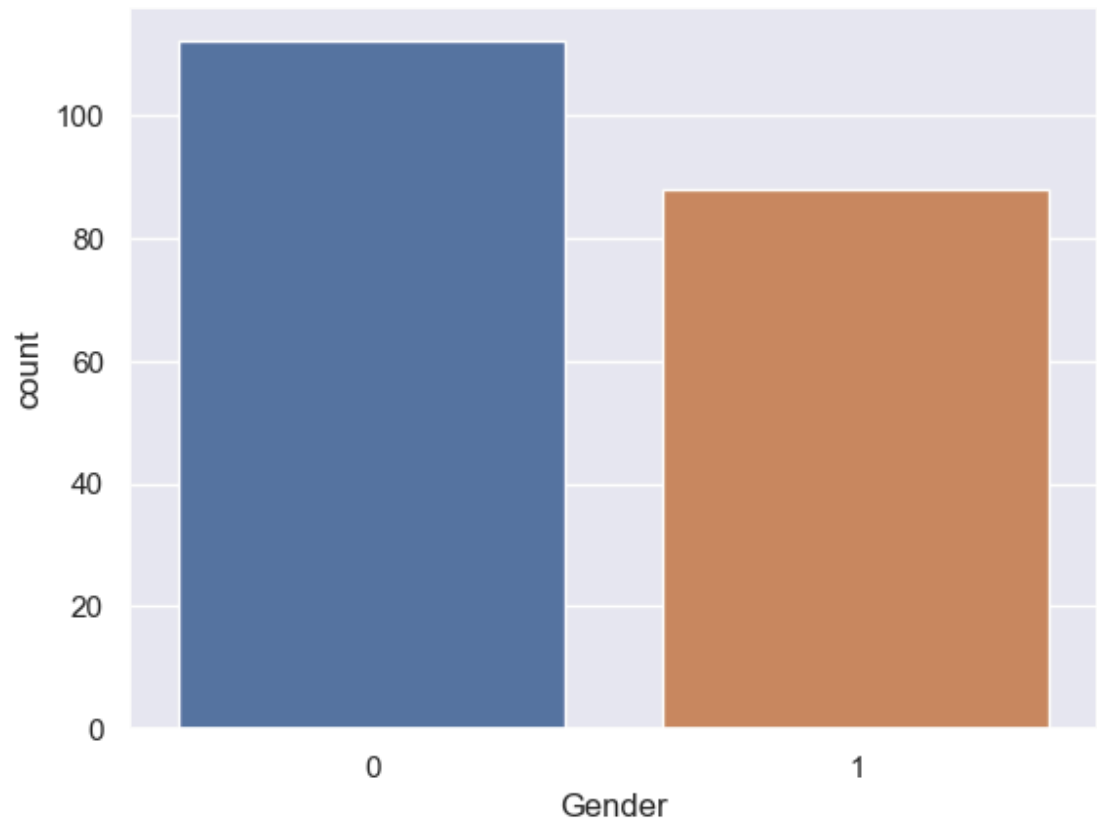
```
In [11]: sns.set()  
sns.pairplot(df)
```

```
Out[11]: <seaborn.axisgrid.PairGrid at 0x26132a0da10>
```



```
In [12]: ▶ sns.countplot(df, x = 'Gender')
```

```
Out[12]: <Axes: xlabel='Gender', ylabel='count'>
```

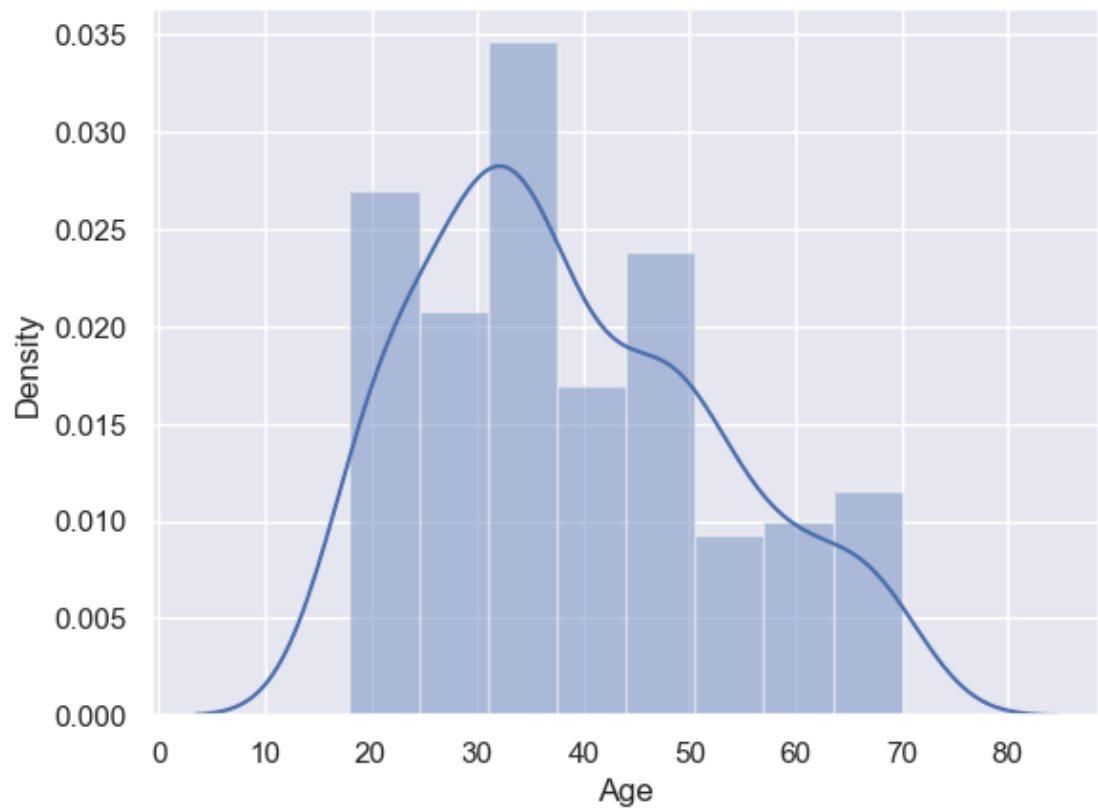


```
In [13]: ▶ print(df.columns)
```

```
Index(['CustomerID', 'Gender', 'Age', 'Annual Income (k$)',  
      'Spending Score (1-100)'],  
      dtype='object')
```

```
In [14]: sns.distplot(df['Age'])
```

```
Out[14]: <Axes: xlabel='Age', ylabel='Density'>
```



```
In [15]: ▶ # Scatter plot of Age vs. Spending Score
sns.scatterplot(x='Annual Income (k$)', y='Spending Score (1-100)', data=c
plt.title('Age vs. Spending Score')
plt.show()
```



```
In [16]: ▶ # Scatter plot of Annual Income vs. Spending Score
sns.scatterplot(x='Annual Income (k$)', y='Spending Score (1-100)', data=c
plt.title('Annual Income vs. Spending Score')
plt.show()
```



Data Modelling

The accuracy of the decision tree model on a test set is crucial. An excellent accuracy score would suggest the model effectively categorizes customers based on the given features.

```
In [17]: ▶ x = df[['Age', 'Annual Income (k$)', 'Gender']]
y = df['Spending Score (1-100)']
```

```
In [18]: ▶ # Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, r

# Create and fit the decision tree model
model = DecisionTreeRegressor()
model.fit(X_train, y_train)

# Predict on the test set
y_pred = model.predict(X_test)

# Assess model accuracy
accuracy = mean_squared_error(y_test, y_pred)
print(f"Model Accuracy: {accuracy}")
```

Model Accuracy: 920.775

```
In [19]: ▶ X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, r
```

```
In [20]: ▶ model = DecisionTreeRegressor(random_state=200)
model.fit(X_train, y_train)
```

Out[20]: DecisionTreeRegressor(random_state=200)

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

```
In [21]: ▶ y_pred = model.predict(X_test)
```

```
In [22]: ▶ accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy}")
```

Model Accuracy: 0.025

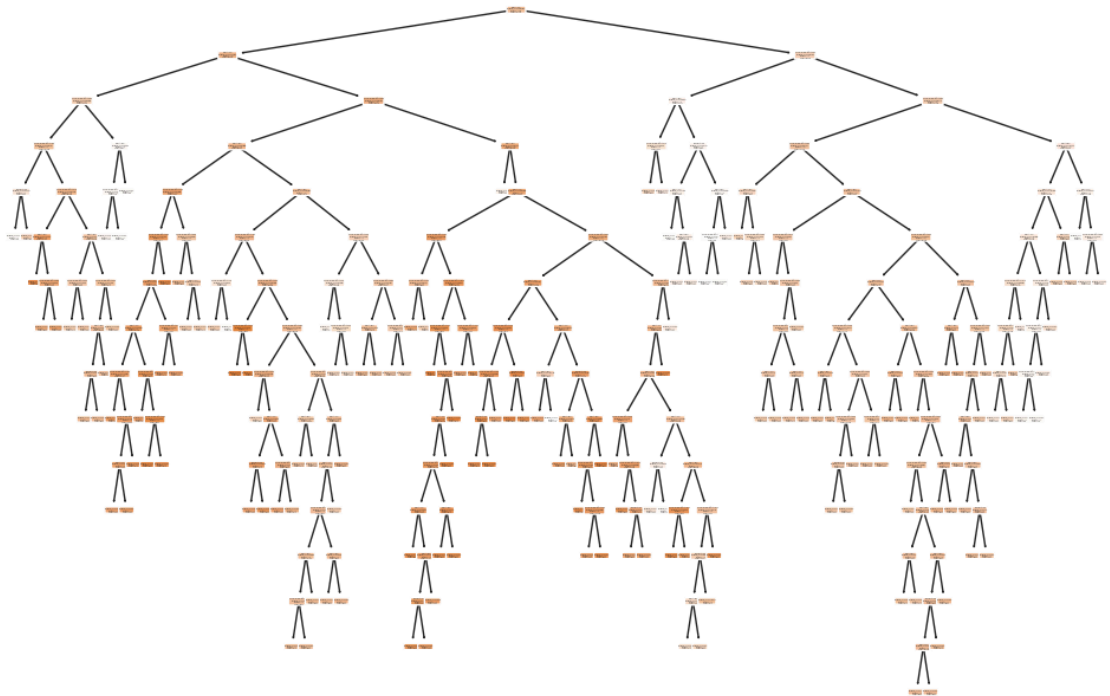
```
In [25]: ▶ mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f'Mean Squared Error: {mse}')
print(f'R-squared: {r2}')
```

Mean Squared Error: 735.875
R-squared: -0.4919207687940963

The decision tree analysis on the dataset featuring customer information like Age, Annual Income, and Spending Score yielded valuable insights for segmenting customers based on their financial profiles and spending habits.

```
In [32]: ▶ plt.figure(figsize=(12, 8))
plot_tree(model, filled=True, feature_names=X.columns.tolist(), rounded=True)
plt.show()
```



```
In [132]: tree_rules = export_text(model, feature_names=list(X.columns))
          print(tree_rules)
```

```

--- Age <= 40.50
|   --- Age <= 20.50
| |   --- Annual Income (k$) <= 69.00
| | |   --- Annual Income (k$) <= 18.50
| | | |   --- Annual Income (k$) <= 15.50
| | | | |   --- value: [39.00]
| | | |   --- Annual Income (k$) > 15.50
| | | | |   --- value: [6.00]
| | |   --- Annual Income (k$) > 18.50
| | | |   --- Annual Income (k$) <= 41.50
| | | | |   --- Age <= 19.00
| | | | | |   --- value: [92.00]
| | | |   --- Age > 19.00
| | | | |   --- Annual Income (k$) <= 29.00
| | | | | |   --- value: [66.00]
| | | | |   --- Annual Income (k$) > 29.00
| | | | | |   --- value: [75.00]
| | |   --- Annual Income (k$) > 41.50
| | | |   --- Age <= 18.50

```

```
In [43]: ▶ # Create Decision Trees with different variations
# Decision Tree 1: Default Parameters (Gini impurity)
dt_gini = DecisionTreeClassifier(random_state=42)
dt_gini.fit(X_train, y_train)
y_pred_gini = dt_gini.predict(X_test)

# Decision Tree 2: Max Depth = 5 (Entropy)
dt_entropy = DecisionTreeClassifier(criterion='entropy', max_depth=5, random_state=42)
dt_entropy.fit(X_train, y_train)
y_pred_entropy = dt_entropy.predict(X_test)

# Decision Tree 3: Min Samples Split = 100 (Gini impurity)
dt_min_samples_split = DecisionTreeClassifier(min_samples_split=100, random_state=42)
dt_min_samples_split.fit(X_train, y_train)
y_pred_min_samples_split = dt_min_samples_split.predict(X_test)
```

```
In [44]: ▶ # Evaluate the models
print("Decision Tree 1 (Gini Impurity) Accuracy:", accuracy_score(y_test, y_pred_gini))
print("Decision Tree 2 (Entropy) Accuracy:", accuracy_score(y_test, y_pred_entropy))
print("Decision Tree 3 (Min Samples Split) Accuracy:", accuracy_score(y_test, y_pred_min_samples_split))

# You can also check other metrics like confusion matrix and classification report
print("Decision Tree 1 (Gini Impurity) Confusion Matrix:\n", confusion_matrix(y_test, y_pred_gini))
print("Decision Tree 2 (Entropy) Confusion Matrix:\n", confusion_matrix(y_test, y_pred_entropy))
print("Decision Tree 3 (Min Samples Split) Confusion Matrix:\n", confusion_matrix(y_test, y_pred_min_samples_split))

print("Decision Tree 1 (Gini Impurity) Classification Report:\n", classification_report(y_test, y_pred_gini))
print("Decision Tree 2 (Entropy) Classification Report:\n", classification_report(y_test, y_pred_entropy))
print("Decision Tree 3 (Min Samples Split) Classification Report:\n", classification_report(y_test, y_pred_min_samples_split))
```

```
Decision Tree 1 (Gini Impurity) Accuracy: 0.0
Decision Tree 2 (Entropy) Accuracy: 0.0
Decision Tree 3 (Min Samples Split) Accuracy: 0.0
Decision Tree 1 (Gini Impurity) Confusion Matrix:
```

```
[[0 0 0 ... 1 0 0]
 [0 0 0 ... 0 0 0]
 [0 1 0 ... 0 0 0]
```

```
...
[0 0 0 ... 0 0 0]
[0 0 0 ... 0 0 0]
[0 0 0 ... 0 0 0]]
```

```
Decision Tree 2 (Entropy) Confusion Matrix:
```

```
[[0 0 0 ... 0 1 0]
 [0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
```

```
...
[0 0 0 ... 0 0 0]
[0 0 0 ... 0 0 0]
[0 0 0 ... 0 0 0]]
```

